Service Overview

ภาพรวมของบริการ (Service Overview)
RescueRequest Service

0. Repository

GitHub: https://github.com/Phattharaphum/rescue-request-service

1. Service Owner

นายภัทรภูมิ กิ่งชัย รหัสนักศึกษา 6609612160 ภาคปกติ

2. Service Purpose

RescueRequest Service เป็นบริการที่รับผิดชอบการจัดการวงจรชีวิตของคำร้องขอความช่วยเหลือจากประชาชน (Citizens) ตั้งแต่การรับเรื่อง การจัดเก็บรายละเอียดที่สำคัญ (เช่น พิกัด, จำนวนคน, ความต้องการพิเศษ) ไปจนถึงการติดตามสถานะการช่วยเหลือ โดยทำหน้าที่เป็นจุดรับข้อมูล (Intake) กลาง เพื่อให้มั่นใจว่าคำร้องทุกรายการถูกบันทึกและสามารถตรวจสอบสถานะได้ตลอดเวลา

3. Pain Point ที่แก้ไข

ในสถานการณ์ภัยพิบัติ ผู้ประสบภัยมักประสบปัญหาในการแจ้งขอความช่วยเหลือเนื่องจากขาดช่องทางที่ระบุรายละเอียดสำคัญ (เช่น พิกัดที่แม่นยำ หรือผู้ป่วยติดเตียง) ได้อย่างครบถ้วนในครั้งเดียว และมักเกิดความกังวลเมื่อไม่ทราบสถานะว่ามีใครรับเรื่องแล้วหรือไม่ นอกจากนี้ ปัญหาเครือข่ายที่ไม่เสถียรทำให้เกิดการส่งข้อมูลซ้ำซ้อน (Duplicate requests) จนเจ้าหน้าที่สับสน บริการนี้จึงถูกออกแบบมาเพื่อรวบรวมข้อมูลให้เป็นระบบ ลดความซ้ำซ้อน และให้ผู้ประสบภัยติดตามความคืบหน้าได้ (Trackable status)

4. Target Users

1. **Citizens (ผู้ประสบภัย):** ส่งคำร้อง + ตรวจสอบสถานะ
2. **Dispatchers / Triage Staff:** คัดกรอง/จัดลำดับความสำคัญ/ปรับสถานะ
3. **Rescue Teams:** ใช้ข้อมูลหน้างานผ่านระบบที่เชื่อมต่อ (เช่น Dispatch/Operations)

5. Service Boundary

- In-scope Responsibilities (สิ่งที่บริการนี้รับผิดชอบ)
  - **Request Ingestion & Validation:**
    รับคำร้อง ตรวจสอบรูปแบบและความครบถ้วนของข้อมูลสำคัญ (Location, Contact, Needs)
  - **State Machine Management:** จัดการสถานะของคำร้องตาม Flow
    SUBMITTED → TRIAGED → ASSIGNED → IN_PROGRESS → RESOLVED หรือ CANCELLED
  - **Duplicate Handling (Idempotency):**
    ตรวจสอบและจัดการคำร้องที่ซ้ำกันจากผู้ใช้คนเดิมในเวลาใกล้เคียงกัน
  - **Status Tracking:** ให้บริการข้อมูลสถานะปัจจุบันแก่ผู้ร้องขอ
  - **Audit Trail / Logs:** เก็บประวัติการเปลี่ยนแปลงสถานะทั้งหมด เพื่อใช้ตรวจสอบย้อนหลัง
- Out-of-scope / Not Responsible For (ไม่รับผิดชอบ)
  - **Resource Dispatching:** การสั่งการหรือจัดสรรยานพาหนะ/ทีมกู้ภัย (เป็นหน้าที่ของ Dispatch Service)
  - **Incident Management:** การจัดการข้อมูลภาพรวมของภัยพิบัติ (เป็นหน้าที่ของ Incident Service)
  - **Medical Diagnosis:** การประเมินอาการทางการแพทย์เชิงลึก
  - **Social Media Scraping/Crawling:** ดึงข้อมูลขอความช่วยเหลือจากโพสต์ในโซเชียลมีเดีย (เช่น Facebook, X) โดยตรง

6. Autonomy / Decision Logic
บริการมีความเป็นอิสระในการตัดสินใจเกี่ยวกับ:

- การยอมรับ (Accept) หรือปฏิเสธ (Drop) คำร้องที่ซ้ำซ้อน (De-duplication logic)
- การตรวจสอบความถูกต้องของรูปแบบข้อมูล (Data Validation) ก่อนบันทึก
- การอนุญาตให้เปลี่ยนสถานะ (State Transition) ว่าเป็นไปตาม Flow ที่กำหนดหรือไม่

หลักการตัดสินใจ (Decision inputs):

6.1 Idempotency (Strong)

- ใช้ X-Idempotency-Key (UUID) จาก client
- ถ้า key เดิมถูกใช้แล้ว → ต้องคืน ผลลัพธ์เดิมเสมอ (TTL 24 ชม.)
- ตอบ 200 OK พร้อม header Idempotency-Replayed: true

6.2 Duplicate Heuristic (Weak, เมื่อไม่มี key)

- ตรวจจับคำร้องซ้ำโดยใช้ uniqueness key:
  incidentId(UUID) + normalizedPhonePrimary + requestType + geoHash(7) + timeBucket(5 minutes)
- ถ้าซ้ำ → ตอบ 409 Conflict พร้อม existingRequestId เพื่อให้ user track รายการเดิม

6.3 Current Status & Completeness

- ถ้า RESOLVED หรือ CANCELLED แล้ว: เป็น terminal state อาจไม่อนุญาตแก้ไข
- ต้องมีข้อมูลจำเป็นครบ: Location, Phone number, peopleCount ≥ 1

บริการสามารถตัดสินใจจัดการข้อมูลซ้ำได้เอง (System decision) โดยไม่ต้องรออนุมัติจากมนุษย์

7. Owned Data

- **Rescue Request Records (Core Domain):** ข้อมูลหลักของคำร้อง ได้แก่ requestId, description, peopleCount, specialNeeds, Location, contact ซึ่งเป็นข้อมูล Core Domain ที่บริการนี้สร้างและดูแลความถูกต้อง
- **Request Lifecycle State:** ข้อมูลสถานะปัจจุบัน (status) และเวลาที่รับเรื่อง (submittedAt) ซึ่งสะท้อนความเป็นไปของคำร้องนั้นๆ
- **Audit Trail:** ประวัติการเปลี่ยนสถานะครบทุกครั้ง

8. Linked Data (Reference Only)

- **incidentId:** อ้างอิงจาก Incident Service เพื่อระบุว่าคำร้องนี้อยู่ภายใต้เหตุการณ์ภัยพิบัติใด (ใช้เพื่อจัดกลุ่มคำร้อง แต่ไม่ได้เก็บรายละเอียดเหตุการณ์ไว้ที่นี่)

9. Non-Functional Requirements

- **Idempotency:** ระบบต้องรองรับการส่งคำร้องซ้ำ (Retry) จากฝั่ง Client กรณีเครือข่ายไม่ดี โดยต้องไม่สร้าง Record ใหม่ แต่ให้คืนค่า requestId เดิมและสถานะล่าสุดกลับไป
- **High Availability:** API สำหรับรับคำร้อง (Create Request) ต้องมีความพร้อมใช้งานสูง รองรับ Load ได้มากในช่วงวิกฤต
- **Data Consistency:** สถานะ (status) ต้องมีความถูกต้อง เพื่อให้ผู้ประสบภัยและเจ้าหน้าที่เห็นข้อมูลตรงกัน
- **Response Time:** การตรวจสอบสถานะ (Get Details) ต้องรวดเร็วเพื่อลดความกังวลของผู้ใช้งาน

10. State Machine Specification

Status Enum (Global):

SUBMITTED, TRIAGED, ASSIGNED, IN_PROGRESS, RESOLVED, CANCELLED

Terminal States:

RESOLVED, CANCELLED

Allowed Transitions

From	To	Allowed Roles	Notes
SUBMITTED	TRIAGED	TRIAGE, DISPATCHER	คัดกรอง + กำหนด priority
TRIAGED	ASSIGNED	DISPATCHER	ต้องมี responderUnitId
ASSIGNED	IN_PROGRESS	RESCUE_TEAM, DISPATCHER	เริ่มปฏิบัติ
IN_PROGRESS	RESOLVED	RESCUE_TEAM, DISPATCHER	จบภารกิจ
SUBMITTED	CANCELLED	DISPATCHER, ADMIN	ต้องมี reason
TRIAGED	CANCELLED	DISPATCHER, ADMIN	ต้องมี reason
ASSIGNED	CANCELLED	DISPATCHER, ADMIN	ต้องมี reason
IN_PROGRESS	CANCELLED	DISPATCHER, ADMIN	ต้องมี reason
RESOLVED / CANCELLED	any	✗	ห้ามเปลี่ยน

Sync Contract

Synchronous Function Contract
Base URL: http://TBD.com

API Contract #1: Create Rescue Request

ข้อมูลทั่วไป

- **Name:** Submit new rescue request
- **Method:** POST
- **Path:** /v1/requests
- **Type:** Synchronous
- **Description:** สร้างคำร้องใหม่ รองรับ idempotency + ตรวจจับ duplicate heuristic

Request

Headers:

```
Content-Type: application/json
X-Idempotency-Key: <UUID> (Optional but recommended)
X-Client-Device: <String> (Optional)
```

Body:

```
{
  "incidentId": "8b9b6d5b-2e6b-4b6e-9b1c-1f8a0c4c2d11",
  "requestType": "flood_rescue",
  "description": "ติดอยู่บนหลังคา น้ำสูง 3 เมตร มีผู้ป่วยติดเตียง",
  "peopleCount": 4,
  "specialNeeds": ["bedridden", "elderly"],
  "location": {
    "lat": 18.7883,
    "lng": 98.9853,
    "floorNumber": "Rooftop",
    "landmark": "ใกล้โรงเรียนบ้านดอน"
  },
  "contact": {
    "name": "สมชาย ใจสู้",
    "phonePrimary": "0812345678",
    "phoneBackup": "0899998888"
  }
}
```

Validation:

- `incidentId`: Required, UUID, ต้องมีอยู่ใน Incident Service
- `peopleCount`: Integer, min 1
- `location.lat`, `location.lng`: Required decimal valid
- `contact.phonePrimary`: Required, 10 digits (normalize ก่อนใช้งาน)

Response

201 Created

```
{
 "requestId": "REQ-8812-9901",
 "incidentId": "8b9b6d5b-2e6b-4b6e-9b1c-1f8a0c4c2d11",
 "status": "SUBMITTED",
 "version": 1,
 "submittedAt": "2026-02-21T10:30:00Z",
 "trackingUrl": "https://help.disaster.com/track/REQ-8812-9901"
}
```

200 OK (Idempotency replay — key ซ้ำ)

- Header: `Idempotency-Replayed: true`

```
{
 "requestId": "REQ-8812-9901",
 "incidentId": "8b9b6d5b-2e6b-4b6e-9b1c-1f8a0c4c2d11",
 "status": "SUBMITTED",
 "version": 1,
 "submittedAt": "2026-02-21T10:30:00Z",
 "trackingUrl": "https://help.disaster.com/track/REQ-8812-9901"
}
```

409 Conflict (Duplicate heuristic)

```
{
 "error": {
  "code": "DUPLICATE_REQUEST",
  "message": "A similar request was submitted recently. Please track the existing request.",
  "existingRequestId": "REQ-8812-9901",
  "traceId": "t-12345"
 }
}
```

400 Bad Request

```
{
 "error": {
   "code": "INVALID_REQUEST",
   "message": "Missing required field: contact.phonePrimary",
   "traceId": "t-12345"
 }
}
```

422 Unprocessable Entity

```
{
 "error": {
   "code": "VALIDATION_FAILED",
   "message": "Field 'peopleCount' must be greater than 0",
   "traceId": "t-12345"
 }
}
```

503 Service Unavailable

```
{
 "error": {
   "code": "INCIDENT_REGISTRY_UNAVAILABLE",
   "message": "Cannot validate incident at the moment. Please retry.",
   "traceId": "t-12345"
 }
}
```

Dependency / Reliability

- DB strong consistency สำหรับสร้าง requestId + เก็บ idempotency mapping

API Contract #2: Get Rescue Request Details

ข้อมูลทั่วไป

- **Name:** Get request details by ID
- **Method:** GET
- **Path:** /v1/requests/{requestId}
- **Type:** Synchronous
- **Description:** ดึงรายละเอียดคำร้องเพื่อให้ citizen ตรวจสอบสถานะ หรือ staff ดูข้อมูล

Request

Path/Query Param

- requestId (String, Required)

Headers

- **Accept:** application/json

Body

- **ไม่มี**

Response

Success: 200 OK


```
{
  "requestId": "REQ-8812-9901",
  "incidentId": "8b9b6d5b-2e6b-4b6e-9b1c-1f8a0c4c2d11",
  "requestType": "flood_rescue",
  "status": "ASSIGNED",
  "submittedAt": "2026-02-21T10:30:00Z",
  "description": "ติดอยู่ชั้น 2 น้ำสูงขึ้นเรื่อยๆ มีผู้ป่วยติดเตียงต้องการอพยพด่วน ออกทางหน้าต่างไม่
ได้",
  "peopleCount": 4,
  "specialNeeds": [
    "bedridden",
    "elderly",
    "medicine_required"
  ],
  "location": {
    "lat": 18.7883,
    "lng": 98.9853,
    "floorNumber": "2",
    "landmark": "บ้านสีฟ้า ตรงข้ามเสาไฟฟ้าต้นที่ 3 ซอย 5 ใกล้โรงเรียนวัดดอน"
  },
  "contact": {
    "name": "สมชาย ใจดี",
    "phonePrimary": "0812345678",
    "phoneBackup": "0899998888"
  },
  "logs": [
    {
      "status": "SUBMITTED",
      "timestamp": "2026-02-21T10:30:00Z",
      "note": "Request received via Mobile App"
    },
    {
      "status": "TRIAGED",
      "timestamp": "2026-02-21T10:45:00Z",
      "note": "Verified urgency: High (Bedridden patient)"
    },
    {
      "status": "ASSIGNED",
      "timestamp": "2026-02-21T11:00:00Z",
      "note": "Team A (Boat Unit) dispatched. ETA 20 mins."
    }
  ]
}
```


Error: 404 Not Found


```
{
  "error": {
    "code": "REQUEST_NOT_FOUND",
    "message": "RequestId does not exist."
  }
}
```

API Contract #3: Search Requests (For Staff)

ข้อมูลทั่วไป

- **Name:** List and filter rescue requests
- **Method:** GET
- **Path:** /v1/requests
- **Type:** Synchronous
- **Description:** staff ดึงรายการคำร้อง โดยกรองเงื่อนไขเพื่อบริหารจัดการ

Request

Query Params

- `incidentId` (UUID, Required)
- `status` (String, Optional, repeatable) เช่น `status=SUBMITTED&status=TRIAGED`
- `needs` (String, Optional, repeatable) เช่น `needs=elderly&needs=bedridden`
- `cursor` (String, Optional)
- `limit` (Int, Default 20, Max 100)

Headers

- `Authorization: Bearer <staff_token>`
- `Accept: application/json`

Response

Success: 200 OK

```
{
  "incidentId": "8b9b6d5b-2e6b-4b6e-9b1c-1f8a0c4c2d11",
  "total": 150,
  "page": 1,
  "items": [
    {
      "requestId": "REQ-8812-9901",
      "status": "SUBMITTED",
      "peopleCount": 4,
      "severity": "HIGH",
      "submittedAt": "2026-02-21T10:30:00Z"
    },
    // ... more items
  ]
}
```

API Contract #4: Update Request Status

ข้อมูลทั่วไป

- **Name:** Update status workflow
- **Method:** PATCH
- **Path:** /v1/requests/{requestId}/status
- **Type:** Synchronous
- **Description:** เปลี่ยนสถานะคำร้องตาม workflow

Request

Path Params

- `requestId` (string, required)

Headers

- `Authorization: Bearer <staff_token>`
- `Content-Type: application/json`

Body

```
{
"targetStatus": "IN_PROGRESS",
"reason": "Rescue boat deployed",
"responderUnitId": "UNIT-BOAT-05",
"meta": {
"estimatedArrivalMinutes": 20
}
}
```

Field Rules

- `targetStatus` (required) enum: `TRIAGED, ASSIGNED, IN_PROGRESS, RESOLVED, CANCELLED`

  หมายเหตุ: ไม่อนุญาตให้ PATCH ไป `SUBMITTED`
- `reason` (optional แต่ **required** เมื่อ `targetStatus = CANCELLED`)
- `responderUnitId`:
  - required เมื่อ `targetStatus = ASSIGNED`
  - ในสถานะอื่น (เช่น IN_PROGRESS/RESOLVED อาจส่งซ้ำเพื่อยืนยัน)
- `meta` optional: ใช้เก็บข้อมูลเพิ่มเติมที่เกี่ยวข้องกับการเปลี่ยนสถานะ (เช่น ETA)

Validation / Logic

1. ตรวจ role/scope:
   - `TRIAGE` เปลี่ยนได้ถึง `TRIAGED`
   - `DISPATCHER` เปลี่ยนได้ `TRIAGED/ASSIGNED/IN_PROGRESS/RESOLVED/CANCELLED`
   - `RESCUE_TEAM` เปลี่ยนได้ `IN_PROGRESS/RESOLVED` (และอาจ `ASSIGNED` เฉพาะกรณีระบบกำหนด)
   - `ADMIN` ได้ทุกอย่าง
   - ไม่ผ่านให้ `403 FORBIDDEN`
2. ตรวจ state machine: ต้องเป็น allowed transitions → ไม่ผ่านให้ `409 STATE_TRANSITION_NOT_ALLOWED`
3. Terminal state: ถ้า current เป็น `RESOLVED` หรือ `CANCELLED` ห้ามเปลี่ยนต่อเสมอ → `409 STATE_TRANSITION_NOT_ALLOWED`

Response

Success: 200 OK

```
{
"requestId": "REQ-8812-9901",
"previousStatus": "ASSIGNED",
"currentStatus": "IN_PROGRESS",
"updatedAt": "2026-02-21T11:15:00Z"
}
```

Error: 422 Unprocessable Entity (State transition invalid)

Async Contract

Asynchronous Function Contract

Message Contract #1: Rescue Request Created

ข้อมูลทั่วไป

- **Message Name:** RescueRequestCreatedEvent
- **Interaction Style:** Event (Pub/Sub)
- **Producer:** RescueRequest Service
- **Consumer:** Dispatch Service (เพื่อจัดรถ), Notification Service (เพื่อส่ง SMS ยืนยัน), Analytics Service
- **Channel/Queue:** rescue.request.created.v1
- **Version:** v1
- **Description:** ส่งทันทีเมื่อสร้างคำร้องสำเร็จ เพื่อให้ระบบปลายทางเริ่มกระบวนการช่วยเหลือ

Request

Message Headers

- `messageType: RescueRequestCreatedEvent`
- `schemaVersion: v1`
- `messageId: <UUID>`
- `traceId: <UUID>`
- `correlationId: REQ-8812-9901`
- `sentAt: ISO-8601`

Message Body

```
{
  "requestId": "REQ-8812-9901",
  "incidentId": "8b9b6d5b-2e6b-4b6e-9b1c-1f8a0c4c2d11",
  "payload": {
    "location": { "lat": 18.7883, "lng": 98.9853 },
    "peopleCount": 4,
    "specialNeeds": ["bedridden"],
  },
  "contact": {
    "phonePrimary": "0812345678"
  }
  "submittedAt": "2026-02-21T10:30:00Z"
}
```

Field Definition:

Field	Type	Required	Description
requestId	String	Y	รหัสคำร้อง (Key หลัก)
incidentId	UUID	Y	รหัสเหตุการณ์
payload	Object	Y	ข้อมูลเนื้อหา
payload.location	Object	Y	พิกัดภูมิศาสตร์
submittedAt	DateTime	N	เวลาที่ส่งคำขอ

Response

- **Message Headers**
  - X-Broker-Ack-Id: String (ID ยืนยันจาก Broker ว่าได้รับข้อความแล้ว)
  - Status: 202 Accepted (กรณีผ่าน HTTP Gateway)
- **Success Message Body**
  - *None / Empty*
  - (หมายเหตุ: การส่งสำเร็จหมายถึง Message Broker ได้รับข้อความและ persist ลง queue แล้ว consumer จะประมวลผลแยกต่างหาก)
- **Reject/Error Message Body**
  - (กรณี Publish ไม่สำเร็จ)

```
{
  "error": {
    "code": "SCHEMA_VALIDATION_FAILED",
    "message": "Field 'peopleCount' must be greater than 0",
    "timestamp": "2026-02-21T11:00:05Z"
  }
}
```

- **Field Definition:**

Field	Type	Required	Description
error.code	String	Y	รหัสข้อผิดพลาดจาก Broker/Gateway
error.message	String	Y	คำอธิบายข้อผิดพลาด
error.timestamp	DateTime	Y	เวลาที่เกิดข้อผิดพลาด

Message Contract #2: Rescue Request Status Changed

ข้อมูลทั่วไป

- **Message Name:** RescueRequestStatusChangedEvent
- **Interaction Style:** Event (Pub/Sub)
- **Producer:** RescueRequest Service
- **Consumer:** Notification Service (แจ้งเตือนประชาชน), Public Dashboard Service
- **Channel/Queue:** rescue.request.status-change.v1
- **Version:** v1
- **Description:** แจ้งเมื่อสถานะเปลี่ยน เพื่อให้ user เห็นความคืบหน้า near real-time

Request

Message Headers

- `messageType: RescueRequestStatusChangedEvent`
- `schemaVersion: v1`
- `messageId: <UUID>`
- `traceId: <UUID>`
- `correlationId: REQ-8812-9901`
- `sentAt: 2026-02-21T11:00:01Z`

Message Body

```
{
  "requestId": "REQ-8812-9901",
  "incidentId": "8b9b6d5b-2e6b-4b6e-9b1c-1f8a0c4c2d11",
  "previousStatus": "TRIAGED",
  "currentStatus": "ASSIGNED",
  "meta": {
    "estimatedArrivalMinutes": 30,
    "responderUnitId": "UNIT-BOAT-05",
    "responderUnitName": "Team A"
  },
  "updatedAt": "2026-02-21T11:00:00Z"
}
```

Field Definition:

Field	Type	Required	Description
requestId	String	Y	รหัสคำร้อง
incidentId	String	Y	รหัส incident ที่คำร้องนี้อ้างอิง
previousStatus	String	Y	สถานะก่อนหน้า (สถานะเดิมก่อนอัปเดต)
currentStatus	String	Y	สถานะใหม่ (TRIAGED, ASSIGNED, IN_PROGRESS, RESOLVED)
meta	Object	N	ข้อมูลเพิ่มเติมที่เกี่ยวข้องกับสถานะใหม่
updatedAt	DateTime	Y	เวลาที่อัปเดตสถานะสำเร็จ (เวลาในระบบ)

Validation Rules

- `currentStatus` ต้องไม่เหมือน `previousStatus`
- `updatedAt` ต้องเป็นปัจจุบันหรืออดีตเท่านั้น

Response

Message Headers

- X-Broker-Ack-Id: String
- Status: 202 Accepted

Success Message Body

None / Empty

(หมายเหตุ: ระบบเป็นแบบ Event-Driven ทางเดียว ผู้ส่ง (Producer) จะไม่รอผลลัพธ์จากผู้รับ (Consumer))

Reject/Error Message Body

(กรณี Publish ไม่สำเร็จ)

```
{
  "error": {
    "code": "PUBLISH_FAILED",
    "message": "Topic 'rescue.request.status-change.v1' not found or not authorized",
    "timestamp": "2026-02-21T11:15:00Z"
  }
}
```

Field Definition:

Field	Type	Required	Description
error.code	String	Y	รหัสข้อผิดพลาด
error.message	String	Y	รายละเอียดปัญหา

Service Data

Service Data

1) Rescue Request Master Data

Field Name	Type	Required	Description	Example
requestId	string	Y	(PK) รหัสคำร้อง	REQ-8812-9901
incidentId	uuid	Y	(Ref) รหัสเหตุการณ์	8b9b6d5b...
requestType	string	Y	ประเภทคำร้อง	flood_rescue
description	text	N	รายละเอียดเพิ่มเติม	ติดอยู่ชั้น 2
peopleCount	integer	Y	จำนวนคน	4
specialNeeds	array	N	ความต้องการพิเศษ	["bedridden"]
latitude	decimal	Y	พิกัดละติจูด	18.7883
longitude	decimal	Y	พิกัดลองจิจูด	98.9853
locationDetails	json	N	ข้อมูลตำแหน่งเพิ่ม	{"floor":"2"}
contactName	string	Y	ชื่อผู้ติดต่อ	สมชาย
contactPhone	string	Y	เบอร์โทรศัพท์	0812345678
submittedAt	datetime	Y	เวลาส่งคำร้อง	2026-02-21T10:30Z

2) Rescue Request Status Events

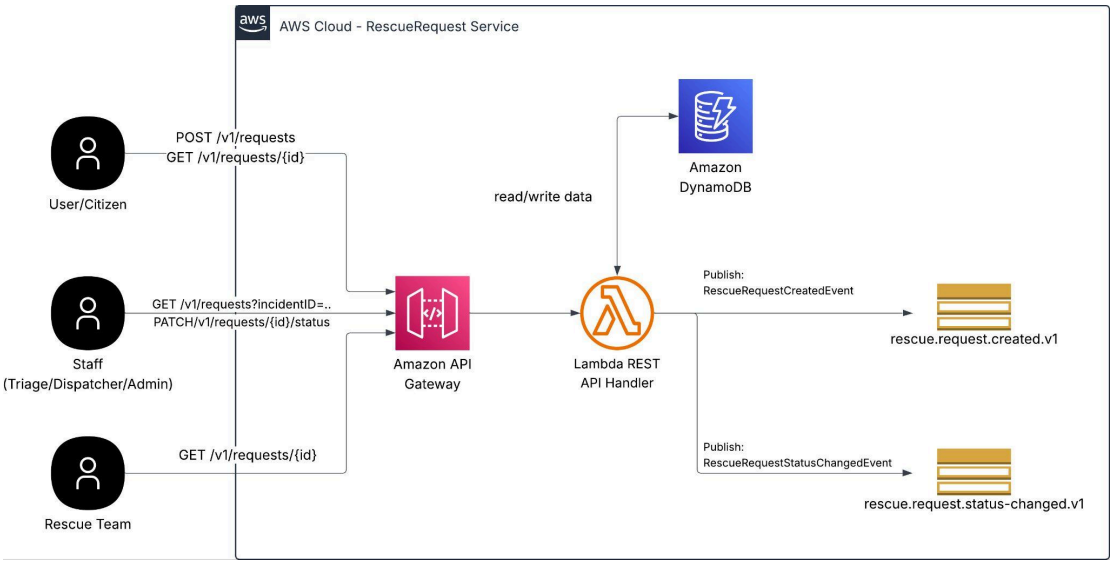
Field Name	Type	Required	Description	Example
eventId	uuid	Y	(PK) ไอดีเหตุการณ์	3f2a...
requestId	string	Y	(FK) อ้างอิงคำร้อง	REQ-8812-9901
previousStatus	enum	N	สถานะก่อนหน้า	SUBMITTED
newStatus	enum	Y	สถานะใหม่ (รวม ASSIGNED)	ASSIGNED
changedBy	string	Y	ผู้กระทำ (System/Staff)	STAFF-009
changedByRole	string	N	Role ผู้กระทำ	DISPATCHER
changeReason	text	N	เหตุผล/หมายเหตุ	เปลี่ยนทีมเนื่องจากรถเสีย
meta	json	N	ข้อมูลเสริม (ETA, Distance)	{"eta_min": 15}
priority_score	int	N	คะแนนความสำคัญ ณ ตอนนั้น	85
responderUnitId	string	N	(สำคัญ) หน่วยที่เกี่ยวข้องใน Event นี้	UNIT-BOAT-05
version	int	Y	เวอร์ชันลำดับเหตุการณ์	3
occurredAt	datetime	Y	เวลาเกิดเหตุการณ์	2026-02-21T11:00Z

3) Rescue Request Current State

Field Name	Type	Required	Description	Example
requestId	string	Y	(PK/FK) รหัสคำร้อง	REQ-8812-9901
incidentId	uuid	Y	รหัสเหตุการณ์	8b9b...
status	enum	Y	สถานะล่าสุด	ASSIGNED
priority_score	int	N	คะแนนล่าสุด	85
priority_level	enum	N	ระดับความสำคัญ	High
assignedUnitId	string	N	หน่วยที่รับผิดชอบปัจจุบัน	UNIT-BOAT-05
assignedAt	datetime	N	เวลาที่เริ่ม Assign ล่าสุด	2026-02-21T11:00Z
latestNote	text	N	หมายเหตุล่าสุด	กำลังเดินทาง
lastUpdatedBy	string	N	แก้ไขล่าสุดโดย	STAFF-009
lastUpdatedAt	datetime	Y	เวลาอัปเดตล่าสุด	2026-02-21T11:00Z

Service Architecture

Service Architecture



Components

Citizen Client (Mobile/Web)

- ส่งคำร้องขอความช่วยเหลือ, ตรวจสอบสถานะด้วย `trackingUrl` / `requestId`

Staff Client (Triage/Dispatcher/Admin Console)

- ค้นหารายการคำร้อง, เปลี่ยนสถานะตาม workflow, ดูประวัติการเปลี่ยนสถานะ

Amazon API Gateway

- จุดรับคำขอแบบ synchronous (REST)
- ทำ throttling/WAF และ route ไปยัง Lambda

Lambda: RescueRequest REST API Handler

- ประมวลผล REST endpoints:
  - `POST /v1/requests` (สร้างคำร้อง + idempotency + duplicate check)
  - `GET /v1/requests/{id}` (ดึงรายละเอียด/สถานะ)
  - `GET /v1/requests?incidentId=...` (staff search)
  - `PATCH /v1/requests/{id}/status` (เปลี่ยนสถานะ + optimistic locking + audit)

Amazon DynamoDB (Owned Data Store)

- เก็บข้อมูลที่บริการเป็นเจ้าของ:
  - **RescueRequests (Master):** รายละเอียดคำร้อง (location/contact/needs/peopleCount)
  - **RescueRequestState (Snapshot):** สถานะล่าสุด + version + assignedUnit + lastUpdated
  - **RescueRequestEvents (Audit Trail):** ประวัติการเปลี่ยนสถานะทั้งหมดแบบ append-only
  - **IdempotencyKeys:** mapping `X-Idempotency-Key → response/requestId` (TTL 24 ชม.)
  - **DuplicateIndex:** เก็บ `dedupeKey → existingRequestId` ช่วงสั้น

Event Bus (Amazon SNS)

- ช่องทางประกาศ event เมื่อ “สร้างคำร้อง” และ “สถานะเปลี่ยน”

Consumer Queues (Amazon SQS per consumer)

- ทำ fan-out จาก SNS ไปยัง SQS ของแต่ละบริการปลายทาง เพื่อความทนทานและรองรับ retry/DLQ

Downstream Consumers

- **Dispatch Service:** รับ `RescueRequestCreatedEvent` เพื่อจัดสรรทีม/ยานพาหนะ
- **Notification Service:** รับ `Created/StatusChanged` เพื่อส่ง SMS/Email/LINE ให้ประชาชน
- **Public Dashboard Service:** อัปเดตสถานะบนหน้า dashboard แบบ near real-time
- **Analytics/Monitoring:** เก็บสถิติ, heatmap, KPI

Explanation

สถาปัตยกรรมของ **RescueRequest Service** แบ่งการทำงานออกเป็น 2 รูปแบบหลัก คือ **synchronous (REST)** และ **asynchronous (event-driven)** เพื่อให้ระบบรับโหลดสูงได้ในช่วงวิกฤตและยังคงติดตามสถานะได้อย่างน่าเชื่อถือ

1) Synchronous (REST) สำหรับงานที่ต้องตอบทันที

ผู้ใช้งาน (Citizen/Staff) เรียกใช้งานผ่าน **Amazon API Gateway** แล้วส่งต่อไปยัง **Lambda REST API Handler** เพื่อประมวลผลคำขอแบบทันที เช่น สร้างคำร้อง, ดูสถานะ, ค้นหารายการ และอัปเดตสถานะ โดย Lambda จะอ่าน/เขียนข้อมูลกับ **DynamoDB** ซึ่งเป็นฐานข้อมูลที่บริการนี้เป็นเจ้าของ

ในการสร้างคำร้อง (POST /v1/requests) ระบบจะทำ **Strong Idempotency** ด้วย X-Idempotency-Key เพื่อรองรับการ retry จากเครือข่ายที่ไม่เสถียร และทำ **Duplicate Heuristic** เมื่อไม่มี key เพื่อลดคำร้องซ้ำ จากนั้นบันทึกข้อมูลลงตาราง Master/Snapshot และเพิ่ม **Audit Trail** แบบ append-only เพื่อให้ตรวจสอบย้อนหลังได้

2) Asynchronous (Event-Driven) สำหรับการกระจายงานไปบริการอื่น

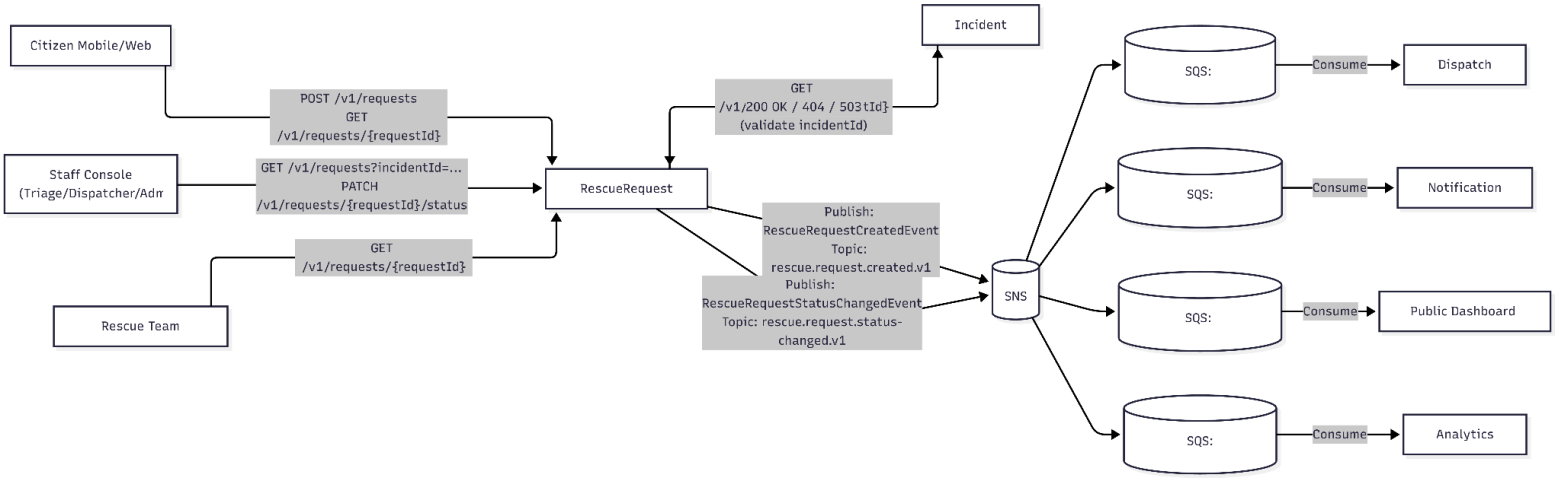
เมื่อสร้างคำร้องสำเร็จหรือมีการเปลี่ยนสถานะ ระบบจะ publish event ไปยัง **SNS/EventBridge** เช่น RescueRequestCreatedEvent และ RescueRequestStatusChangedEvent เพื่อให้บริการปลายทางเริ่มทำงานต่อได้

เพื่อความทนทาน จึงทำ fan-out ไปยัง **SQS ต่อ consumer** ทำให้แต่ละบริการ (Dispatch/Notification/Dashboard/Analytics) สามารถรับ message ตามความเร็วของตนเอง มี retry และ DLQ ได้ ลดผลกระทบเมื่อบางบริการล่มหรือช้า

ด้วยการแยก **REST path** สำหรับการรับเรื่อง/ตรวจสอบสถานะ และ **Event path** สำหรับการส่งต่อผลลัพธ์และการแจ้งเตือน สถาปัตยกรรมนี้จึงช่วยให้ RescueRequest Service เป็น “ศูนย์กลางรับคำร้องที่เชื่อถือได้” รองรับโหลดสูง ลดข้อมูลซ้ำ และทำให้ทุกฝ่ายติดตามความคืบหน้าได้อย่างต่อเนื่องในสถานการณ์ภัยพิบัติ

Service Interaction

Service Interaction



Upstream Services (บริการต้นทางที่เรียกใช้งาน RescueRequest Service)

- **Citizen Mobile/Web App (Citizens)**
  เรียกสร้างคำร้องและตรวจสอบสถานะ
  - POST /v1/requests
  - GET /v1/requests/{requestId}

- **Staff Console (Triage/Dispatcher/Admin)**
  เรียกค้นหา/บริหารคำร้องและเปลี่ยนสถานะ
  - GET /v1/requests?incidentId=...
  - PATCH /v1/requests/{requestId}/status

- **Rescue Team App/System**
  เรียกดูรายละเอียด/สถานะเพื่อปฏิบัติงานหน้างาน
  - GET /v1/requests/{requestId}

Downstream Services (บริการปลายทางที่ RescueRequest Service เรียกหรือส่งข้อมูลไป)

A) Downstream แบบ Synchronous

- **Incident Service**
  ใช้ตรวจสอบ/อ้างอิงว่า incidentId ถูกต้องและมีอยู่จริง (registry/validation)

B) Downstream แบบ Asynchronous (ส่งข้อมูลผ่าน Pub/Sub)

- **Event Bus (SNS)**: ช่องทางกระจาย event
  - rescue.request.created.v1 (RescueRequestCreatedEvent)
  - rescue.request.status-changed.v1 (RescueRequestStatusChangedEvent)

- **Dispatch Service (ผ่าน SQS: dispatch.inbox.v1)**
  รับ event เพื่อเริ่มกระบวนการจัดสรรทีม/ยานพาหนะ

- **Notification Service (ผ่าน SQS: notification.inbox.v1)**
  รับ event เพื่อส่ง SMS/Email/LINE แจ้งประชาชนเรื่องการรับเรื่อง/สถานะเปลี่ยน

- **Public Dashboard Service (ผ่าน SQS: dashboard.inbox.v1)**
  รับ status-changed เพื่ออัปเดต dashboard แบบ near real-time

- **Analytics Service (ผ่าน SQS: analytics.inbox.v1)**
  รับ event เพื่อทำสถิติ/รายงาน/heatmap และการติดตาม KPI

Dependency Mapping

Dependency Mapping – RescueRequest Service

1) Incident Service

- **Type:** Service
- **Interaction Style:** Synchronous (REST)
- **Purpose:** ตรวจสอบความถูกต้อง/การมีอยู่ของ `incidentId` ก่อนบันทึกคำร้อง (ใช้เป็น registry/reference)
- **Criticality:** Critical
- **Failure Handling:**
  - หาก Incident Service ไม่ตอบสนอง/timeout → ตอบ `503 INCIDENT_REGISTRY_UNAVAILABLE` ให้ client retry

2) Amazon DynamoDB

- **Type:** Database
- **Interaction Style:** Internal Data Access (read/write)
- **Purpose:** จัดเก็บข้อมูลที่ RescueRequest Service เป็นเจ้าของ
  - RescueRequests (Master)
  - RescueRequestState (Snapshot/Current Status)
  - RescueRequestEvents (Audit Trail)
  - IdempotencyKeys (TTL 24h)
  - DuplicateIndex (TTL short)
- **Criticality:** Critical
- **Failure Handling:**
  - ใช้ **DynamoDB Transaction** เพื่อให้ master/state/audit เขียนสอดคล้องกัน
  - หาก write/transaction ล้มเหลว → ไม่สร้างคำร้อง/ไม่เปลี่ยนสถานะ และไม่ publish event

3) Amazon DynamoDB – IdempotencyKeys

- **Type:** Database (DynamoDB Table)
- **Interaction Style:** Internal Data Access (read/write)
- **Purpose:** รองรับ **Strong Idempotency** ด้วย `X-Idempotency-Key` ให้ replay ผลลัพธ์เดิมได้ (TTL 24 ชั่วโมง)
- **Criticality:** Critical
- **Failure Handling:**
  - หากอ่าน/เขียน idempotency mapping ล้มเหลว → ปฏิเสธการสร้างคำร้อง (ตอบ 5xx) เพื่อกันการสร้างซ้ำ
  - ใช้ conditional write/atomic put เพื่อลด race condition

4) Amazon DynamoDB – DuplicateIndex

- **Type:** Database (DynamoDB Table)
- **Interaction Style:** Internal Data Access (read/write)
- **Purpose:** รองรับ **Weak Duplicate Heuristic** เมื่อไม่มี idempotency key เพื่อตรวจจับคำร้องซ้ำในช่วงเวลาสั้น
- **Criticality:** Non-Critical
- **Failure Handling:**
  - หากตาราง dedupe ใช้งานไม่ได้/อ่านไม่สำเร็จ → ระบบยัง "รับคำร้องได้" แต่เสี่ยงเกิด duplicate มากขึ้น
  - หากเขียน dedupe key ไม่สำเร็จ → ไม่ block การสร้างคำร้อง

5) Amazon SNS (rescue.request.created.v1)

- **Type:** Topic / Event Bus
- **Interaction Style:** Asynchronous (Event Pub/Sub)
- **Purpose:** กระจายเหตุการณ์ "สร้างคำร้องสำเร็จ" ให้บริการปลายทางเริ่มทำงาน (Dispatch/Notification/Analytics)
- **Criticality:** Non-Critical (ต่อการตอบ REST) / Critical (ต่อ workflow ปลายทาง)
- **Failure Handling:**
  - หาก publish ไม่สำเร็จ → บันทึก log/metric และ retry

6) Amazon SNS (rescue.request.status-changed.v1)

- **Type:** Topic / Event Bus
- **Interaction Style:** Asynchronous (Event Pub/Sub)
- **Purpose:** แจ้งการเปลี่ยนสถานะ (TRIAGED/ASSIGNED/IN_PROGRESS/RESOLVED/CANCELLED) ให้ Notification และ Dashboard อัปเดต near real-time
- **Criticality:** Non-Critical (ต่อการเปลี่ยนสถานะในฐานข้อมูล)
- **Failure Handling:**
    - หาก publish ไม่สำเร็จ → retry/log และให้ state ใน DB เป็น source of truth (client ยัง query GET ได้)

7) Amazon SQS (dispatch.inbox.v1)

- **Type:** Queue
- **Interaction Style:** Asynchronous (Event Delivery / Fan-out Queue)
- **Purpose:** บัฟเฟอร์ event ให้ Dispatch Service รับไปจัดสรรทีม/ยานพาหนะโดยไม่กระทบ RescueRequest Service
- **Criticality:** Non-Critical (ต่อการสร้างคำร้อง) / Critical (ต่อการ dispatch)
- **Failure Handling:**
    - ใช้ retry ของ consumer และ DLQ เมื่อประมวลผลล้มเหลวซ้ำ

8) Amazon SQS (notification.inbox.v1)

- **Type:** Queue
- **Interaction Style:** Asynchronous (Event Delivery / Fan-out Queue)
- **Purpose:** ส่ง event ให้ Notification Service เพื่อแจ้งประชาชน (SMS/Email/LINE)
- **Criticality:** Non-Critical
- **Failure Handling:**
    - consumer retry + DLQ
    - หากส่งไม่ออก → ไม่กระทบความถูกต้องของสถานะในระบบหลัก (ผู้ใช้ยังเช็คสถานะผ่าน GET ได้)

9) Amazon SQS (dashboard.inbox.v1)

- **Type:** Queue
- **Interaction Style:** Asynchronous (Event Delivery)
- **Purpose:** ส่ง status-changed ให้ Public Dashboard Service อัปเดตภาพรวม/near real-time
- **Criticality:** Non-Critical
- **Failure Handling:**
    - retry + DLQ
    - dashboard สามารถ rebuild จากการ query ข้อมูลใหม่ (eventual consistency)

10) Amazon SQS (analytics.inbox.v1)

- **Type:** Queue
- **Interaction Style:** Asynchronous (Event Delivery)
- **Purpose:** ส่งข้อมูล event ให้ Analytics pipeline เพื่อสถิติและการวิเคราะห์
- **Criticality:** Non-Critical
- **Failure Handling:**
    - retry + DLQ
    - หากล้มเหลวให้เก็บไว้ในคิว/ส่ง DLQ แล้ว reprocess ภายหลัง